CHAPTER 3

PRINCIPLES OF PROGRAMMING AND PROBLEM SOLVING

CHAPTER ESSENCE AT A GLANCE:
Systematic program development transforms complex real-world requirements into verified executable logic through structured problem analysis, modular algorithmic design, and rigorous testing.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Phased Program Development Lifecycle:** Developing robust software requires sequential adherence to problem analysis, algorithmic design, flowchart visualization, language coding, translation and debugging, and documentation.

• **Algorithmic Rigor and Finiteness:** An effective algorithm must accept clear inputs, maintain unambiguous precision across every instruction, terminate after a finite number of steps, and generate correct outputs.

• **Standardized Flowchart Conventions:** Visual flowcharts employ standardized geometric symbols such as ovals for boundaries, parallelograms for input and output, rectangles for operations, and rhombuses for conditional branches.

• **Defect Classification and Debugging Methodology:** Program defects span compile-time syntax violations, runtime abnormal terminations such as division by zero, and silent logical flaws yielding incorrect results.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Lifecycle} = \text{Problem Analysis} \rightarrow \text{Algorithm Design} \rightarrow \text{Coding} \rightarrow \text{Translation} \rightarrow \text{Debugging} \rightarrow \text{Documentation}$$

Six Systematic Phases of Program Development

$$\text{Algorithm Criteria} = \text{Input} + \text{Definiteness} + \text{Finiteness} + \text{Effectiveness} + \text{Output}$$

Knuth Five Mandatory Algorithm Criteria

$$\text{Flowchart Elements} = \text{Terminal (Oval)} + \text{Process (Rect)} + \text{I/O (Parallelogram)} + \text{Decision (Rhombus)}$$

Standard Geometric Flowchart Symbols

$$\text{Defect Space} = \text{Syntax Errors (Compile)} + \text{Logical Errors (Output)} - \text{Runtime Errors (Crashes)}$$

Three-Tier Program Defect Classification

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Syntax errors are caught by the compiler, but logical errors compile cleanly and produce incorrect outputs without warning.

• **Alert:** Every algorithm must terminate after a finite number of steps; infinite loops violate the finiteness property.

• **Alert:** In flowcharts, decision rhombuses must have exactly one incoming flowline and at least two outgoing conditional branches (True and False).

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. List and explain the six distinct phases in systematic program development. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Problem Analysis and Algorithm Design: Understanding user requirements, defining input-output constraints, and designing step-by-step algorithms and flowcharts. **[1 Mark]**

• Coding, Translation, Debugging, and Documentation: Translating logic into source code, compiling, eliminating errors, and writing developer documentation. **[1 Mark]**

2. State four essential characteristics of an effective algorithm. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Finiteness: The algorithm must terminate after a finite number of execution steps under all conditions. **[0.5 Mark]**

• Definiteness: Each step must be precisely defined, clear, and completely unambiguous. **[0.5 Mark]**

• Input and Output: It must accept zero or more valid inputs and produce at least one defined output. **[0.5 Mark]**

• Effectiveness: All operations must be basic enough to be carried out exactly in a finite amount of time. **[0.5 Mark]**

3. Draw and explain the standard flowchart symbols for input and output, processing, and decision making. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Parallelogram (Input and Output Symbol): Represents data entry operations from keyboard or data display operations to screen. **[1 Mark]**

• Rectangle (Processing Symbol): Represents arithmetic calculations, data transformations, and variable assignments. **[1 Mark]**

• Rhombus or Diamond (Decision Symbol): Evaluates a conditional expression with two possible exit paths (Yes/No or True/False). **[1 Mark]**

4. Write a step-by-step algorithm to find the largest among three distinct numbers A, B, and C. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Step 1: Start. Step 2: Read numerical values A, B, and C. **[1 Mark]**

• Step 3: Compare A and B. If A is greater than B, compare A and C: if A is greater than C, then set Big = A, else set Big = C. **[1 Mark]**

• Step 4: Else (B is greater than or equal to A), compare B and C: if B is greater than C, then set Big = B, else set Big = C. Step 5: Print Big. Step 6: Stop. **[1 Mark]**

5. Differentiate between syntax errors, logical errors, and runtime errors with illustrative examples. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Syntax Errors: Violations of the grammatical rules of the programming language detected during compilation (e.g. omitting a semicolon, misspelling a keyword). **[1 Mark]**

• Logical Errors: Flaws in the algorithmic design where the program compiles cleanly but computes incorrect outputs (e.g. writing sum = a - b instead of sum = a + b). **[1 Mark]**

• Runtime Errors: Illegal operations detected during program execution that cause abrupt termination or crashes (e.g. division by zero, accessing an invalid array index). **[1 Mark]**

6. Explain the concept of Top-Down design in modular programming and its practical benefits. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Top-Down Modular Decomposition: The software engineering methodology of decomposing a large complex problem into smaller, manageable subproblems or functional modules. **[1.5 Marks]**

• Practical Benefits: Enhances code readability, allows parallel development by team members, simplifies testing and debugging, and promotes reusable function libraries. **[1.5 Marks]**

7. What is debugging? Describe the systematic techniques used to detect and eliminate program errors. **[Score: 4 Marks]**

Answer & Key Valuation Steps:

• Definition of Debugging: The diagnostic process of locating, analyzing, and correcting defects and errors inside a computer program. **[1 Mark]**

• Desk Checking: Manually tracing the code on paper with sample input data and recording variable values step by step. **[1 Mark]**

• Diagnostic Print Statements: Temporarily inserting output statements to print intermediate variable states during execution. **[1 Mark]**

• Integrated Debugger Tools: Utilizing breakpoints, step-by-step single stepping, and memory watch windows in modern IDEs to inspect execution flow. **[1 Mark]**